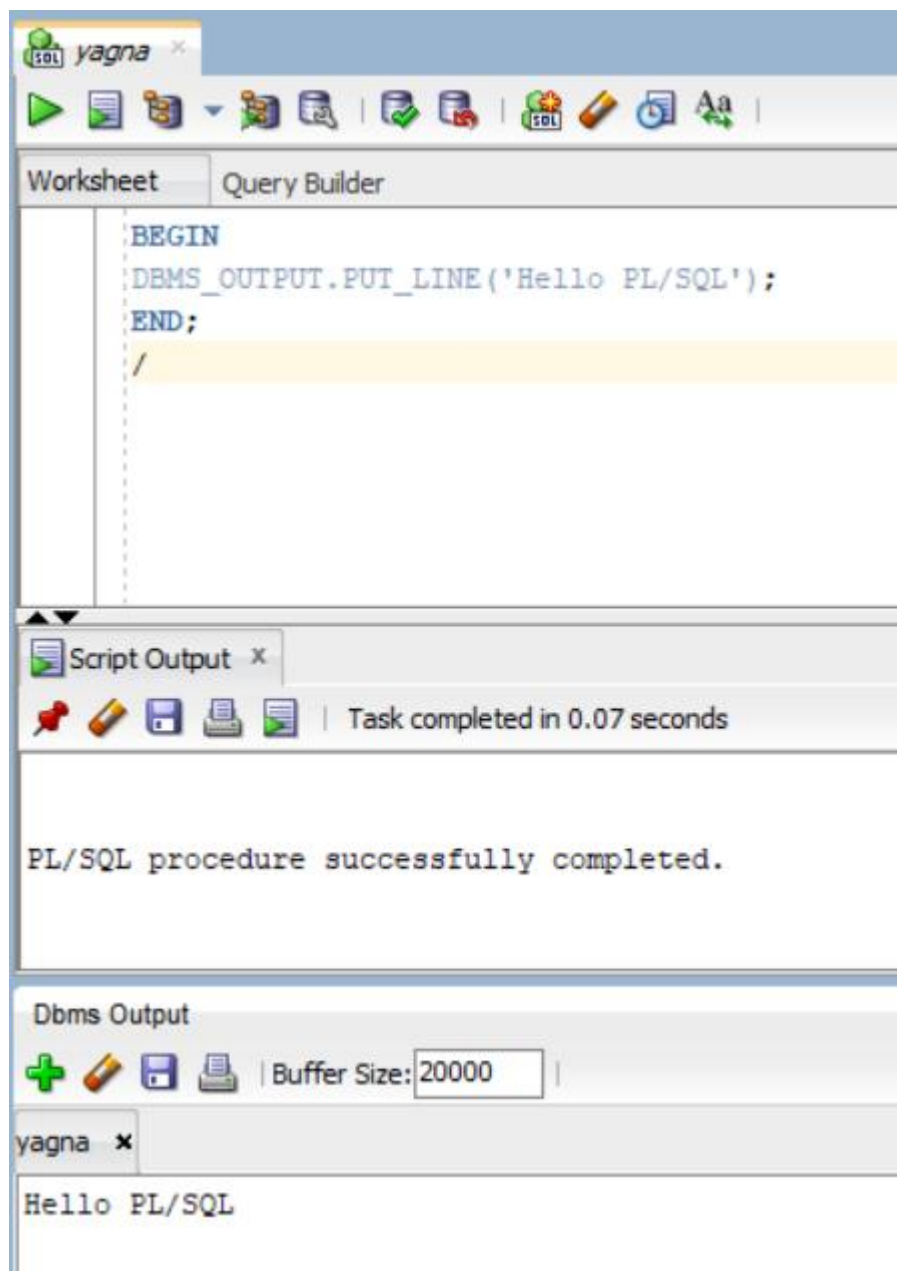
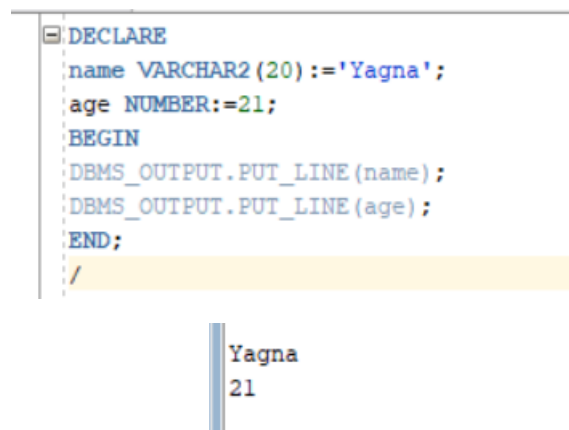


Module 3 - pl/sql programming

1. Hello program



2. Variables & data types



3. Constants

The screenshot shows the Oracle SQL Developer interface. The 'Worksheet' tab is active, displaying a PL/SQL script:

```
DECLARE
pi CONSTANT NUMBER:=3.14159;
BEGIN
DBMS_OUTPUT.PUT_LINE(pi);
END;
```

The script is executed, and the 'Script Output' window shows the message: 'Task completed in 0.049 seconds'. Below this, the 'Dbms Output' window shows the output: '3.14159'.

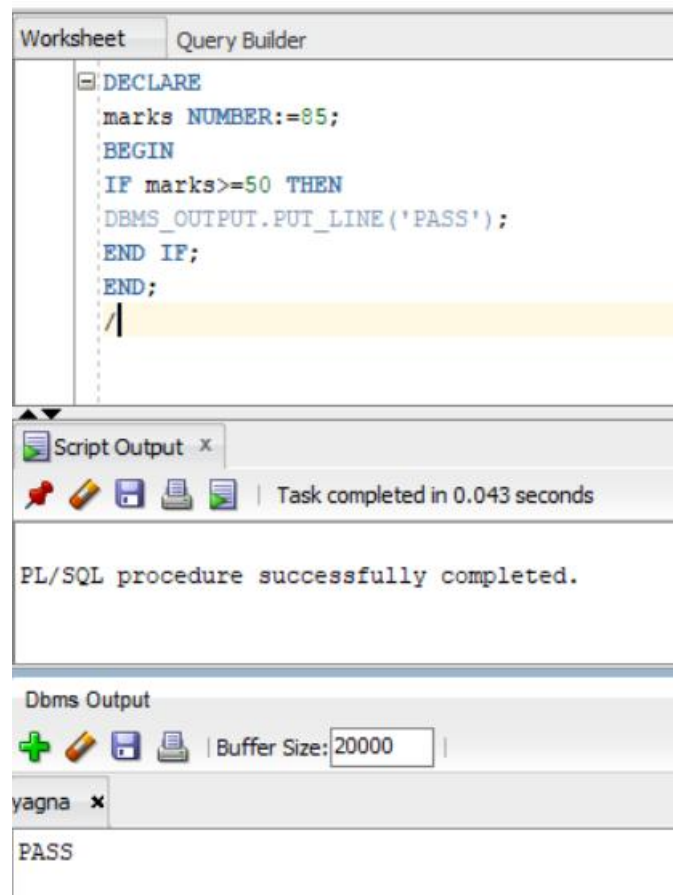
4. Accepting variable values

The screenshot shows the Oracle SQL Developer interface. The 'Worksheet' tab is active, displaying a PL/SQL script:

```
DECLARE
a NUMBER:=10;
b NUMBER:=20;
c NUMBER;
BEGIN
c:=a+b;
DBMS_OUTPUT.PUT_LINE(c);
END;
```

The script is executed, and the 'Script Output' window shows the message: 'Task completed in 0.045 seconds'. Below this, the 'Dbms Output' window shows the output: '30'.

5. If statement



The screenshot shows the Oracle SQL Developer interface. The 'Query Builder' tab is active, displaying a PL/SQL block with an IF statement. The code declares a variable 'marks' with a value of 85 and prints 'PASS' because 85 is greater than or equal to 50. The 'Script Output' window shows the task completed in 0.043 seconds. The 'Dbms Output' window shows the output 'PASS'.

```
DECLARE
marks NUMBER:=85;
BEGIN
IF marks>=50 THEN
DBMS_OUTPUT.PUT_LINE('PASS');
END IF;
END;
/
```

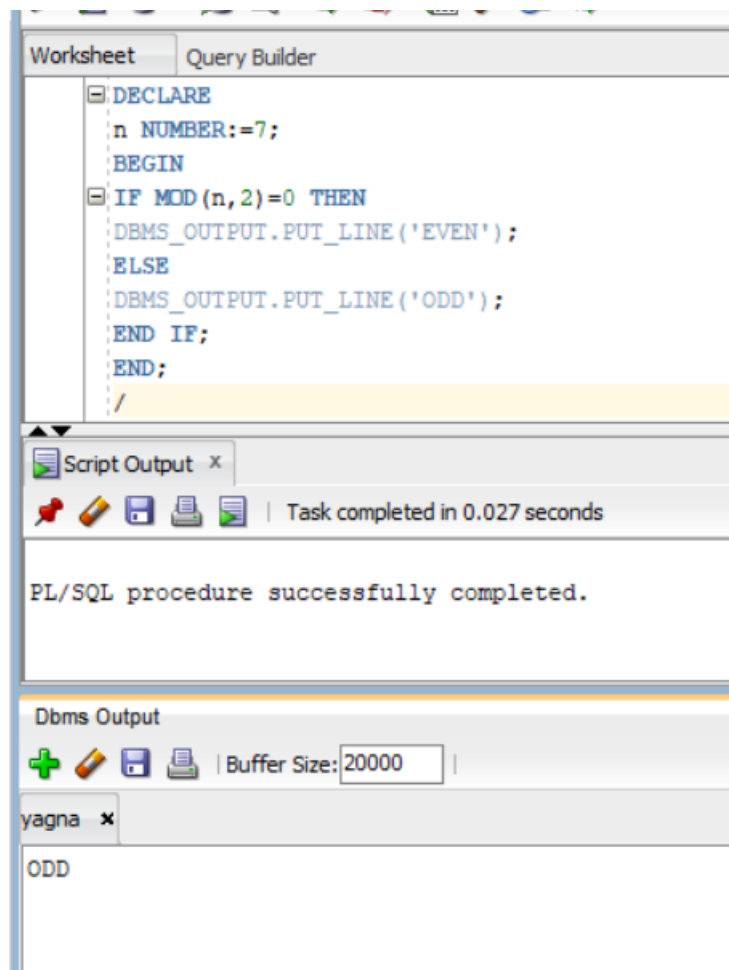
Script Output x
Task completed in 0.043 seconds

PL/SQL procedure successfully completed.

Dbms Output
Buffer Size: 20000

yagna x
PASS

6. If else



The screenshot shows the Oracle SQL Developer interface. The 'Query Builder' tab is active, displaying a PL/SQL block with an IF-ELSE statement. The code declares a variable 'n' with a value of 7 and prints 'ODD' because 7 is not divisible by 2. The 'Script Output' window shows the task completed in 0.027 seconds. The 'Dbms Output' window shows the output 'ODD'.

```
DECLARE
n NUMBER:=7;
BEGIN
IF MOD(n,2)=0 THEN
DBMS_OUTPUT.PUT_LINE('EVEN');
ELSE
DBMS_OUTPUT.PUT_LINE('ODD');
END IF;
END;
/
```

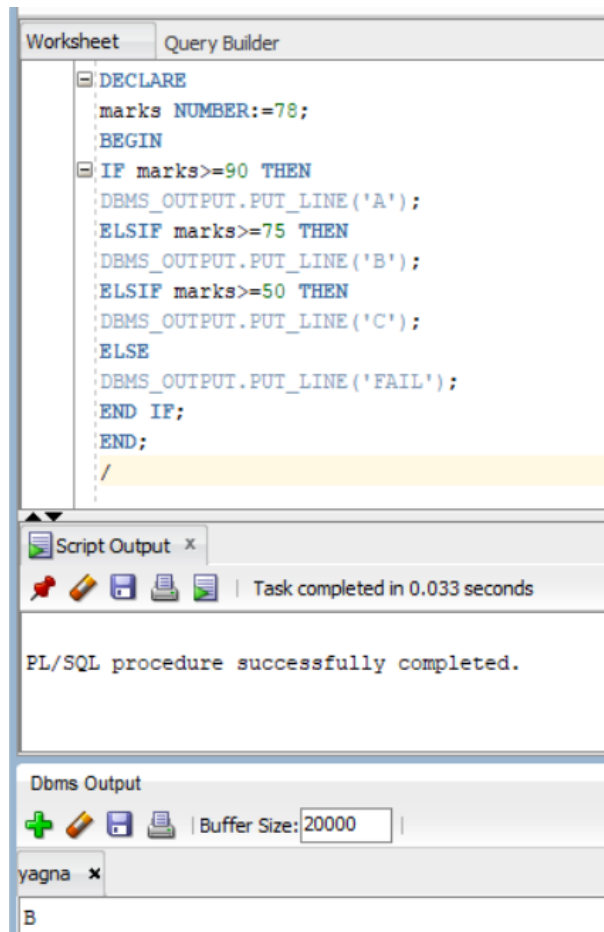
Script Output x
Task completed in 0.027 seconds

PL/SQL procedure successfully completed.

Dbms Output
Buffer Size: 20000

yagna x
ODD

7. If elsif else

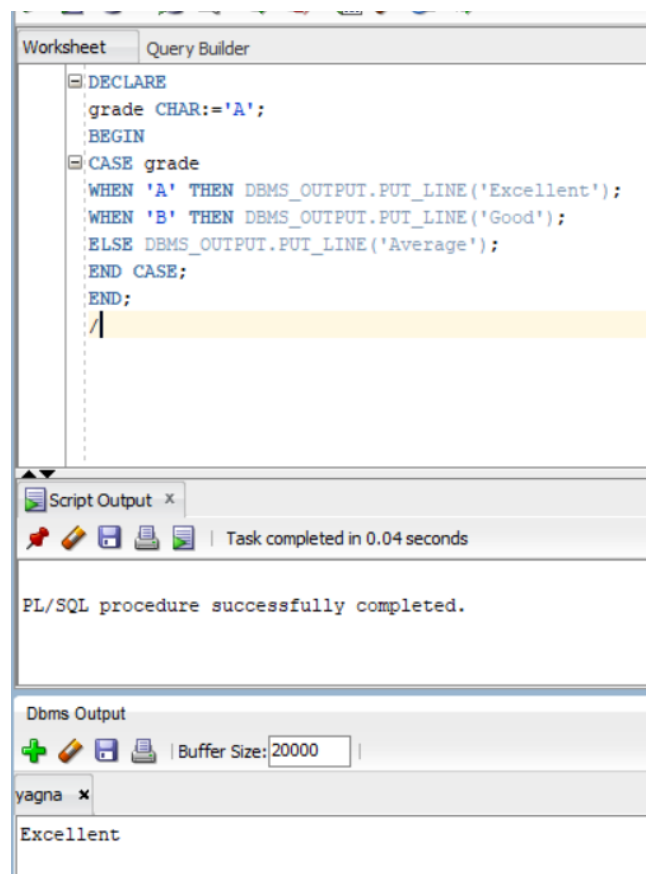


The screenshot displays the SQL Developer interface. The top pane, titled 'Query Builder', contains the following PL/SQL code:

```
DECLARE
marks NUMBER:=78;
BEGIN
IF marks>=90 THEN
DBMS_OUTPUT.PUT_LINE('A');
ELSIF marks>=75 THEN
DBMS_OUTPUT.PUT_LINE('B');
ELSIF marks>=50 THEN
DBMS_OUTPUT.PUT_LINE('C');
ELSE
DBMS_OUTPUT.PUT_LINE('FAIL');
END IF;
END;
```

The bottom pane shows the 'Script Output' window with the message 'Task completed in 0.033 seconds' and 'PL/SQL procedure successfully completed.' Below this, the 'Dbms Output' window shows the output 'B'.

8. Case statement

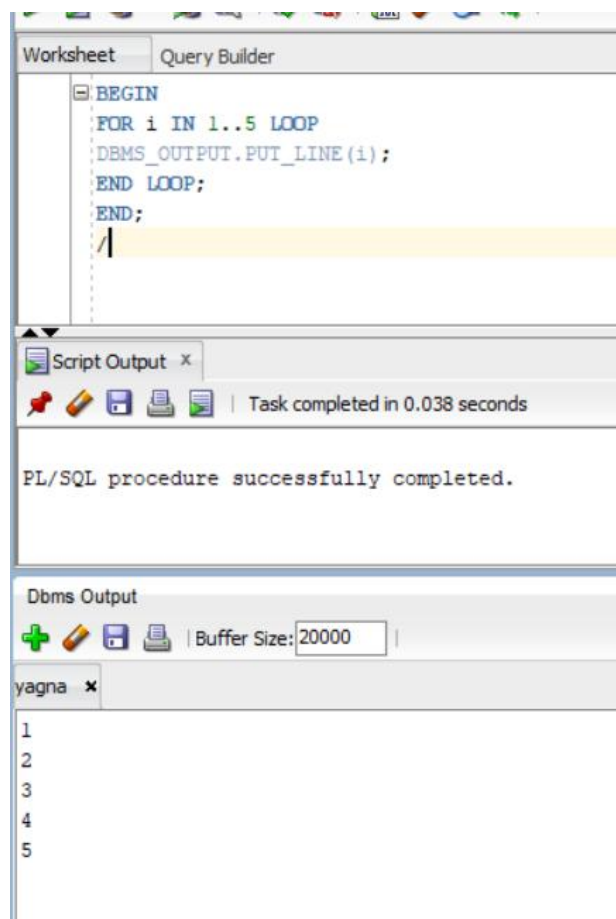


The screenshot displays the SQL Developer interface. The top pane, titled 'Query Builder', contains the following PL/SQL code:

```
DECLARE
grade CHAR:='A';
BEGIN
CASE grade
WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Excellent');
WHEN 'B' THEN DBMS_OUTPUT.PUT_LINE('Good');
ELSE DBMS_OUTPUT.PUT_LINE('Average');
END CASE;
END;
```

The bottom pane shows the 'Script Output' window with the message 'Task completed in 0.04 seconds' and 'PL/SQL procedure successfully completed.' Below this, the 'Dbms Output' window shows the output 'Excellent'.

9. For loop

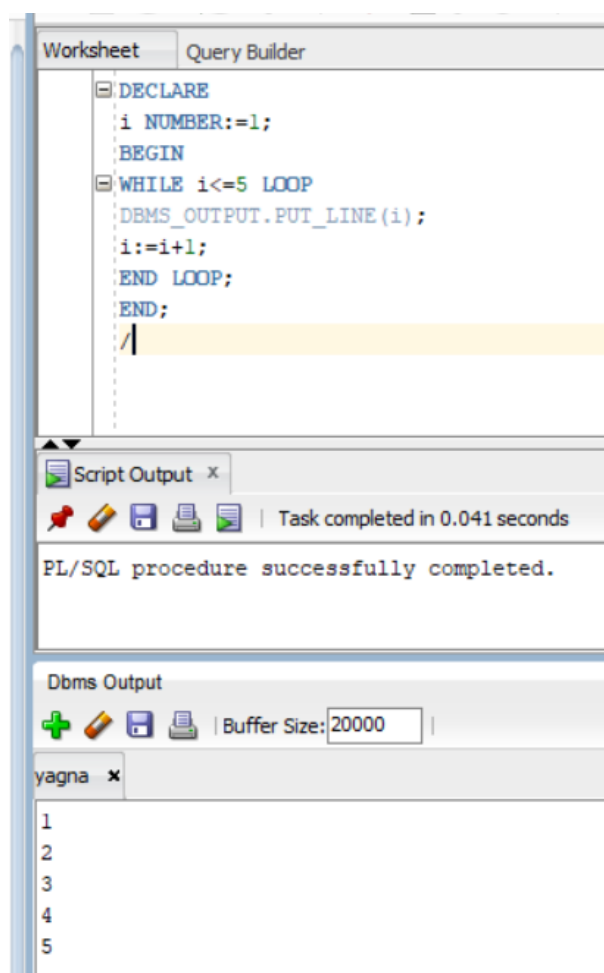


The screenshot displays the SQL Developer interface. The 'Query Builder' tab is active, showing a PL/SQL script with a for loop. The script is as follows:

```
BEGIN
FOR i IN 1..5 LOOP
  DBMS_OUTPUT.PUT_LINE(i);
END LOOP;
END;
```

Below the script, the 'Script Output' window shows the message: 'Task completed in 0.038 seconds' and 'PL/SQL procedure successfully completed.' The 'Dbms Output' window is also visible, showing the output of the loop: the numbers 1 through 5, each on a new line.

10. While loop

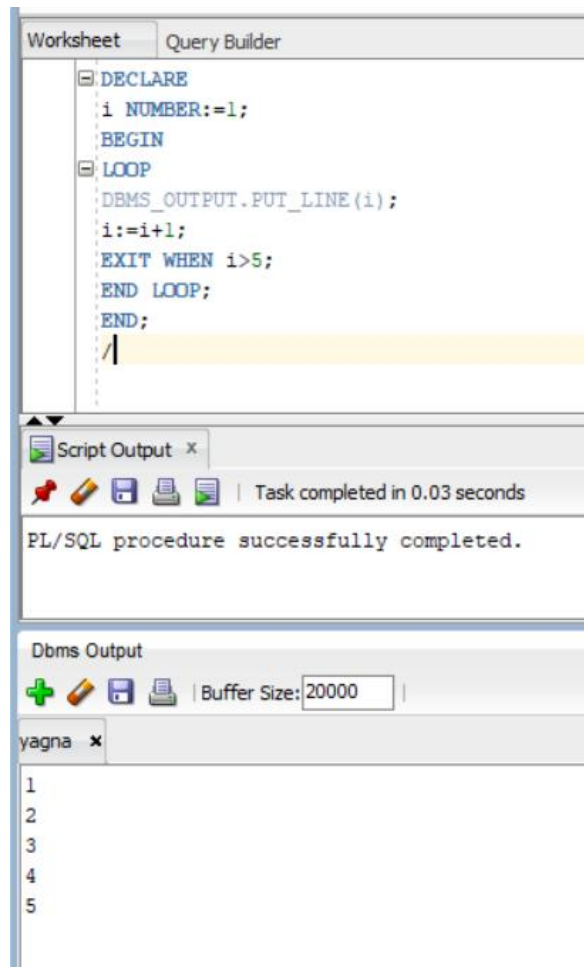


The screenshot displays the SQL Developer interface. The 'Query Builder' tab is active, showing a PL/SQL script with a while loop. The script is as follows:

```
DECLARE
  i NUMBER:=1;
BEGIN
  WHILE i<=5 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
    i:=i+1;
  END LOOP;
END;
```

Below the script, the 'Script Output' window shows the message: 'Task completed in 0.041 seconds' and 'PL/SQL procedure successfully completed.' The 'Dbms Output' window is also visible, showing the output of the loop: the numbers 1 through 5, each on a new line.

11. Simple loop

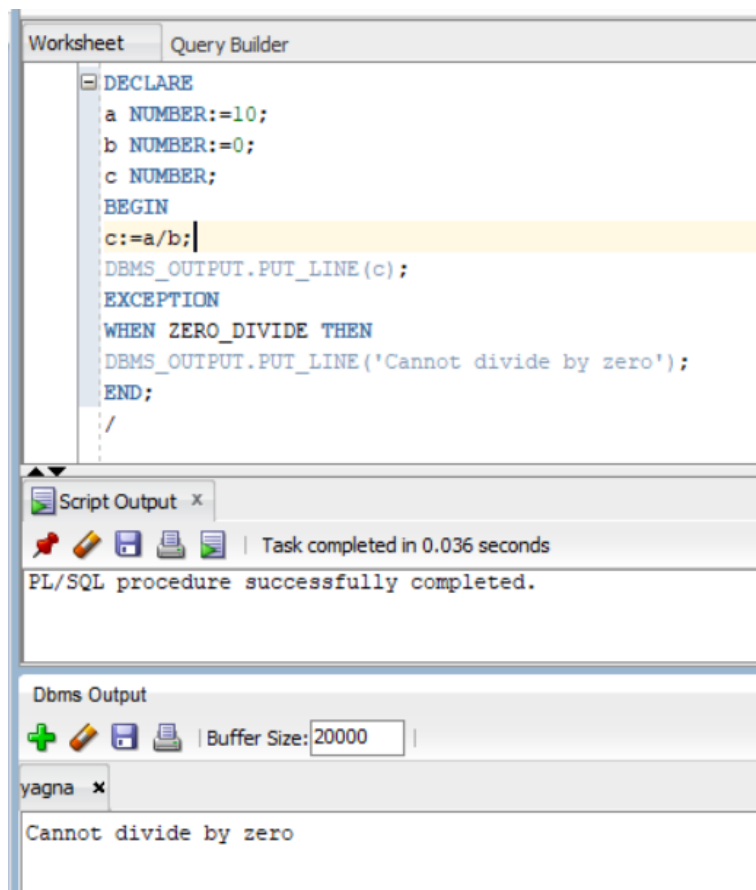


The screenshot shows the SQL Developer interface. The 'Query Builder' tab is active, displaying a PL/SQL procedure with a simple loop. The code is as follows:

```
DECLARE
i NUMBER:=1;
BEGIN
LOOP
DBMS_OUTPUT.PUT_LINE(i);
i:=i+1;
EXIT WHEN i>5;
END LOOP;
END;
```

Below the code editor, the 'Script Output' window shows the message: 'Task completed in 0.03 seconds' and 'PL/SQL procedure successfully completed.' The 'Dbms Output' window shows the output of the loop, displaying the numbers 1 through 5 on separate lines.

12. Predefined exception



The screenshot shows the SQL Developer interface. The 'Query Builder' tab is active, displaying a PL/SQL procedure that demonstrates a predefined exception. The code is as follows:

```
DECLARE
a NUMBER:=10;
b NUMBER:=0;
c NUMBER;
BEGIN
c:=a/b;
DBMS_OUTPUT.PUT_LINE(c);
EXCEPTION
WHEN ZERO_DIVIDE THEN
DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');
END;
```

Below the code editor, the 'Script Output' window shows the message: 'Task completed in 0.036 seconds' and 'PL/SQL procedure successfully completed.' The 'Dbms Output' window shows the output of the procedure, displaying the message 'Cannot divide by zero'.

13. User defined exception

The screenshot displays the SQL Developer interface with the 'Query Builder' tab active. The SQL editor contains the following code:

```
DECLARE
age NUMBER:=15;
invalid_age EXCEPTION;
BEGIN
IF age<18 THEN
RAISE invalid_age;
END IF;
EXCEPTION
WHEN invalid_age THEN
DBMS_OUTPUT.PUT_LINE('Not Eligible');
END;
/
```

Below the editor, the 'Script Output' window shows the message: 'Task completed in 0.029 seconds' and 'PL/SQL procedure successfully completed.' The 'Dbms Output' window, with a buffer size of 20000, shows the output: 'Not Eligible'.

14. Stored procedure

The screenshot displays the SQL Developer interface with the 'Query Builder' tab active. The SQL editor contains the following code:

```
CREATE OR REPLACE PROCEDURE greet
IS
BEGIN
DBMS_OUTPUT.PUT_LINE('Welcome');
END;
/

BEGIN
greet;
END;
/
```

Below the editor, the 'Script Output' window shows the message: 'Task completed in 0.027 seconds' and 'PL/SQL procedure successfully completed.' The 'Dbms Output' window, with a buffer size of 20000, shows the output: 'Welcome'.

15. Create table

The screenshot shows the YAGNA SQL editor interface. The top toolbar includes icons for running, saving, and other standard SQL IDE functions. The main window is titled 'Worksheet' and 'Query Builder'. The SQL script is as follows:

```
CREATE TABLE emp(  
  empno NUMBER PRIMARY KEY,  
  ename VARCHAR2(30),  
  job VARCHAR2(30),  
  sal NUMBER,  
  deptno NUMBER  
);  
  
INSERT INTO emp VALUES(101,'Yagna','Manager',50000,10);  
INSERT INTO emp VALUES(102,'Rahul','Developer',40000,20);  
INSERT INTO emp VALUES(103,'Sneha','Tester',35000,30);  
INSERT INTO emp VALUES(104,'Ravi','Clerk',25000,10);  
  
COMMIT;  
  
SELECT * FROM emp;
```

Below the script, the 'Script Output' and 'Query Result' tabs are visible. The 'Query Result' tab shows the output of the SELECT statement:

	EMPNO	ENAME	JOB	SAL	DEPTNO
1	101	Yagna	Manager	50000	10
2	102	Rahul	Developer	40000	20
3	103	Sneha	Tester	35000	30
4	104	Ravi	Clerk	25000	10

The screenshot shows the YAGNA SQL editor with a PL/SQL procedure script:

```
DECLARE  
  CURSOR c IS  
  SELECT ename,sal FROM emp;  
  
  v_name emp.ename%TYPE;  
  v_sal emp.sal%TYPE;  
  
BEGIN  
  OPEN c;  
  
  LOOP  
    FETCH c INTO v_name,v_sal;  
    EXIT WHEN c%NOTFOUND;  
  
    DBMS_OUTPUT.PUT_LINE(v_name||' - '||v_sal);  
  
  END LOOP;  
  
  CLOSE c;  
END;  
/
```

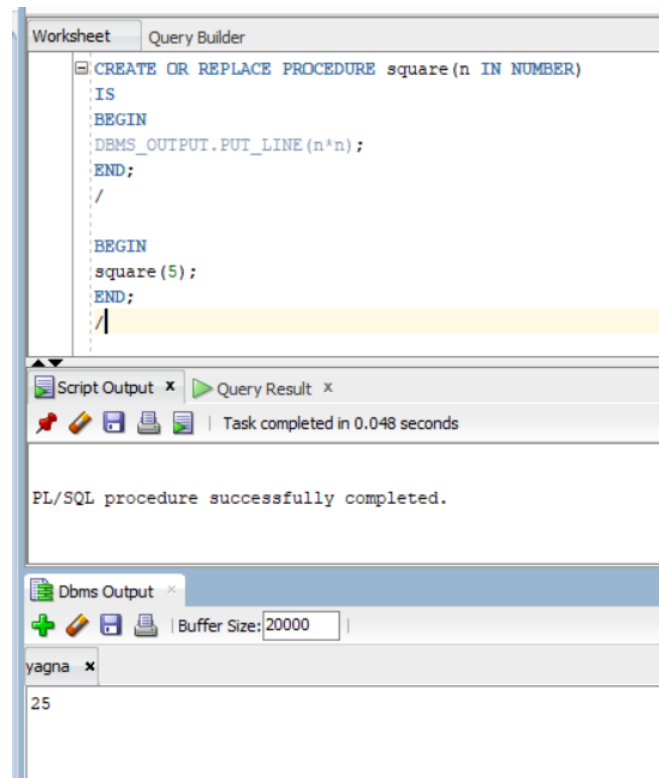
The 'Script Output' and 'Query Result' tabs are visible. The 'Query Result' tab shows the output of the PL/SQL procedure:

```
PL/SQL procedure successfully completed.
```

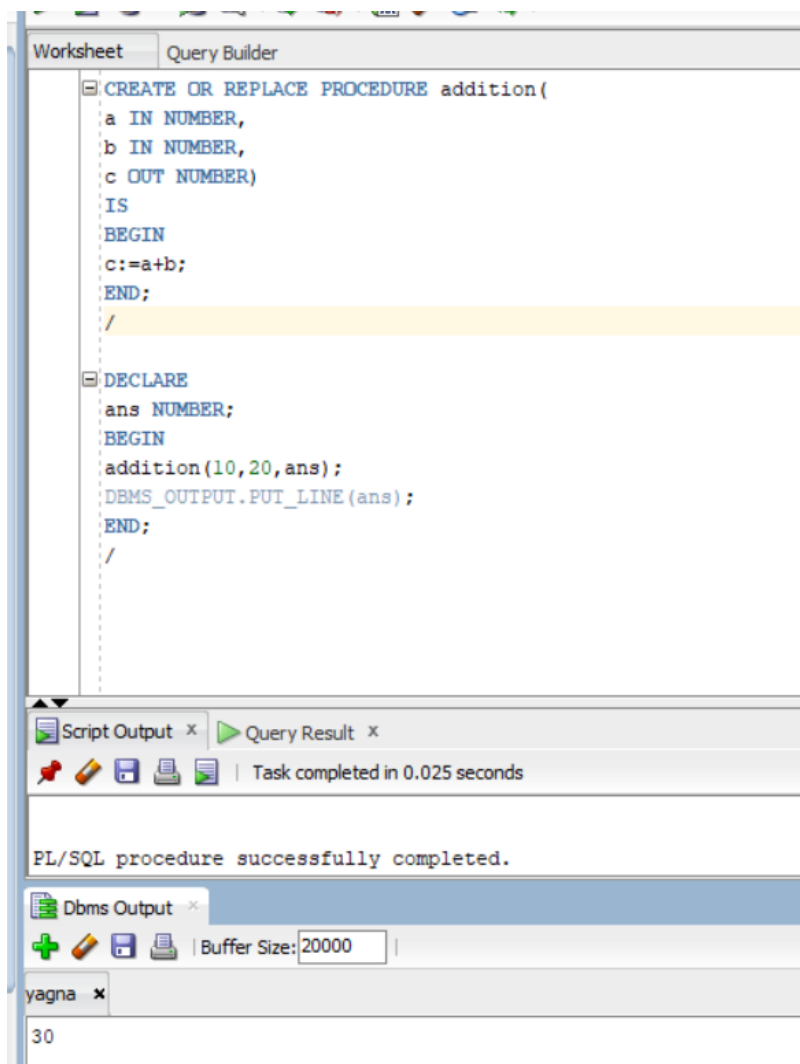
Below the script, the 'Dbms Output' tab is visible, showing the output of the DBMS_OUTPUT.PUT_LINE statement:

```
yagna x  
Yagna - 50000  
Rahul - 40000  
Sneha - 35000  
Ravi - 25000
```


16. Procedure with in parameter



17. Procedure with out parameter



18. Function

The screenshot displays a database IDE interface with the following components:

- Worksheet / Query Builder:** Contains the following PL/SQL code:

```
CREATE OR REPLACE FUNCTION sq(n NUMBER)
RETURN NUMBER
IS
BEGIN
RETURN n*n;
END;
/

BEGIN
DBMS_OUTPUT.PUT_LINE(sq(6));
END;
/
```
- Script Output / Query Result:** Shows the status "Task completed in 0.032 seconds".
- Dbms Output:** Displays the message "PL/SQL procedure successfully completed." with a buffer size of 20000.
- yagna:** A status bar at the bottom showing the page number "36".

19. Package

The screenshot displays the SQL Developer interface with the 'Query Builder' tab active. The main window contains the following SQL script:

```
-- package specification
CREATE OR REPLACE PACKAGE math_pkg IS
FUNCTION add_num(a NUMBER,b NUMBER) RETURN NUMBER;
END;
/

-- package body
CREATE OR REPLACE PACKAGE BODY math_pkg IS
FUNCTION add_num(a NUMBER,b NUMBER)
RETURN NUMBER
IS
BEGIN
RETURN a+b;
END;
END;
/

-- execute
BEGIN
DBMS_OUTPUT.PUT_LINE(math_pkg.add_num(5,6));
END;
/
```

Below the script, the 'Script Output' window shows the execution results:

Task completed in 0.028 seconds

Package MATH_PKG compiled

Package Body MATH_PKG compiled

PL/SQL procedure successfully completed.

The 'Dbms Output' window is also visible, showing a buffer size of 20000. The 'yagna' window shows the output '11'.

20. BEFORE INSERT trigger

The screenshot displays the SQL Developer interface with the 'Query Builder' tab active. The main window contains the following SQL script:

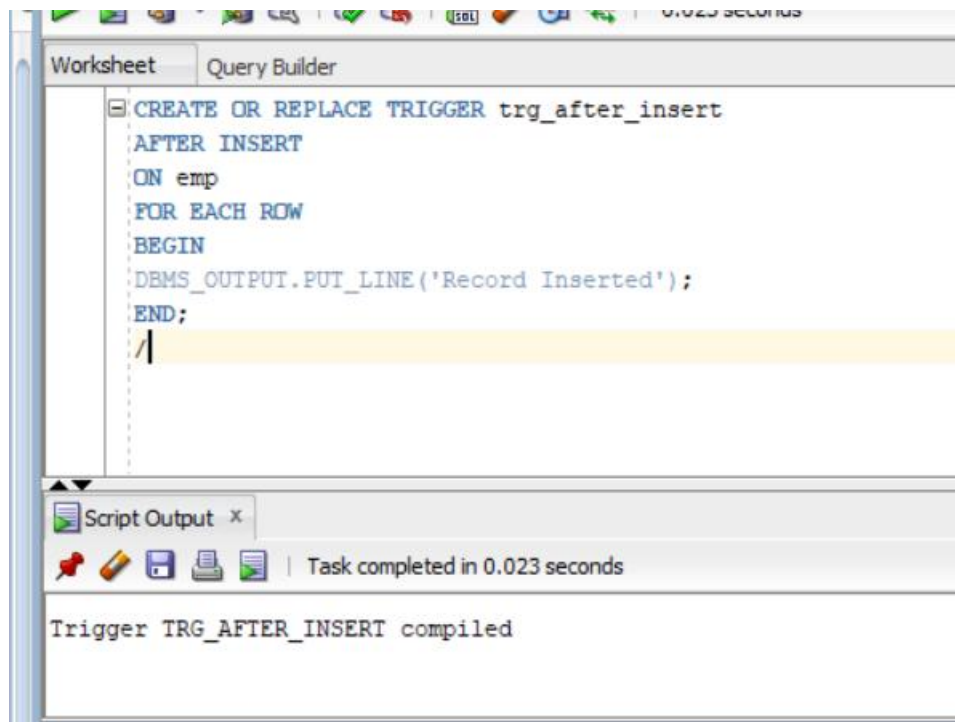
```
CREATE OR REPLACE TRIGGER trg_before_insert
BEFORE INSERT
ON emp
FOR EACH ROW
BEGIN
DBMS_OUTPUT.PUT_LINE('Record Inserting');
END;
/
```

Below the script, the 'Script Output' window shows the execution results:

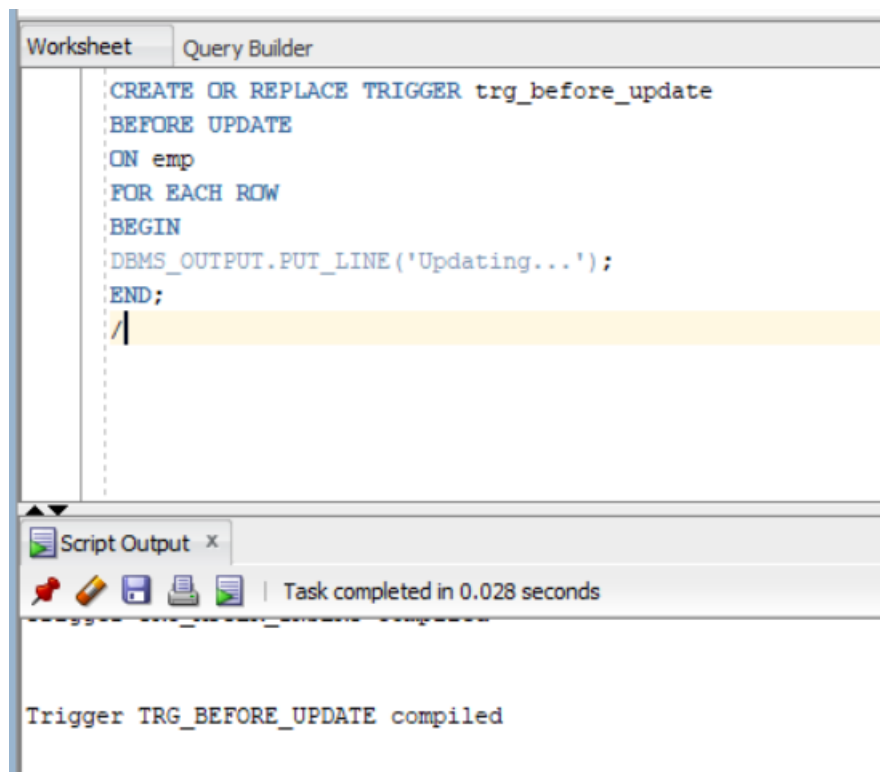
Task completed in 0.04 seconds

Trigger TRG_BEFORE_INSERT compiled

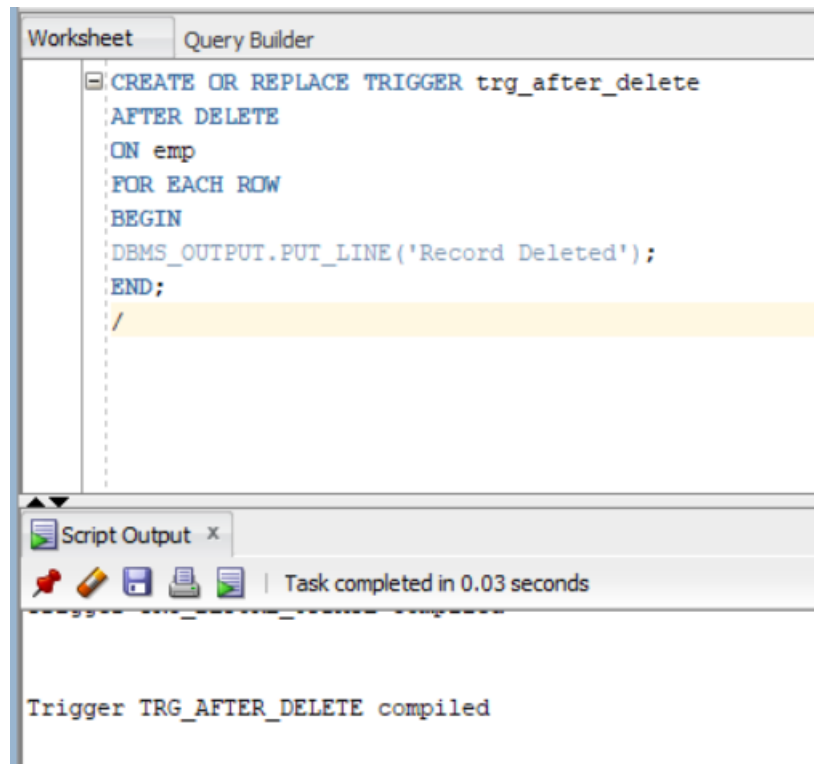
21. AFTER INSERT trigger



22. BEFORE UPDATE Trigger



23. AFTER DELETE Trigger

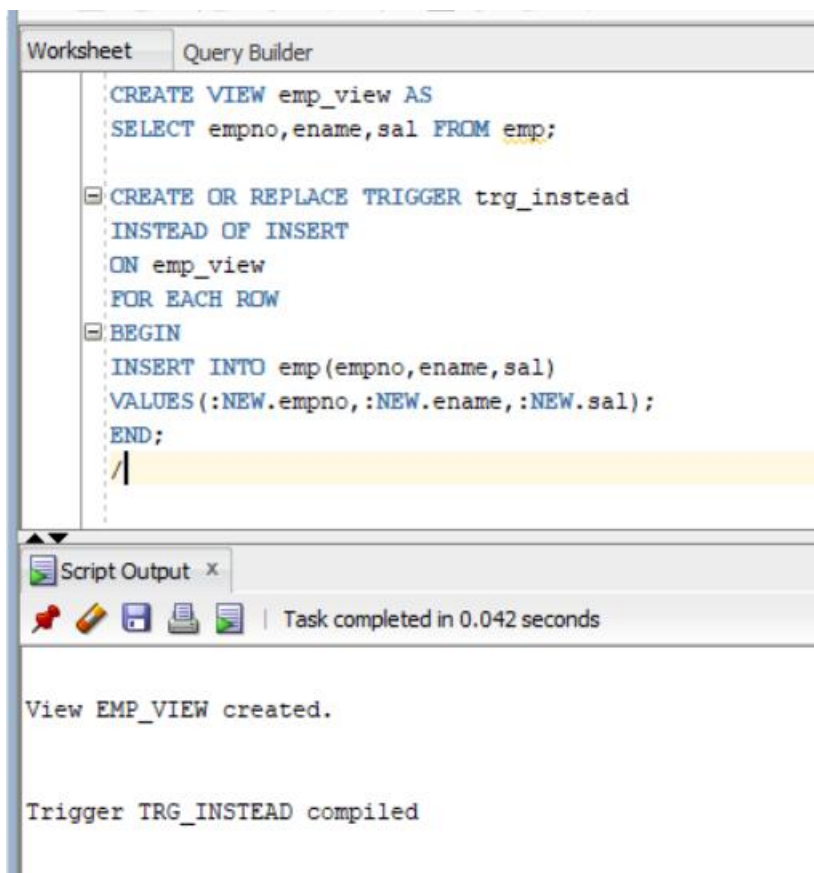


The screenshot shows the SQL Developer interface with the 'Query Builder' tab active. The SQL editor contains the following code:

```
CREATE OR REPLACE TRIGGER trg_after_delete
AFTER DELETE
ON emp
FOR EACH ROW
BEGIN
  DBMS_OUTPUT.PUT_LINE('Record Deleted');
END;
/
```

Below the editor, the 'Script Output' window shows the message: 'Trigger TRG_AFTER_DELETE compiled'. The task completion time is 0.03 seconds.

24. INSTEAD OF Trigger (on View)



The screenshot shows the SQL Developer interface with the 'Query Builder' tab active. The SQL editor contains the following code:

```
CREATE VIEW emp_view AS
SELECT empno,ename,sal FROM emp;

CREATE OR REPLACE TRIGGER trg_instead
INSTEAD OF INSERT
ON emp_view
FOR EACH ROW
BEGIN
  INSERT INTO emp(empno,ename,sal)
VALUES (:NEW.empno,:NEW.ename,:NEW.sal);
END;
/
```

Below the editor, the 'Script Output' window shows two messages: 'View EMP_VIEW created.' and 'Trigger TRG_INSTEAD compiled'. The task completion time is 0.042 seconds.